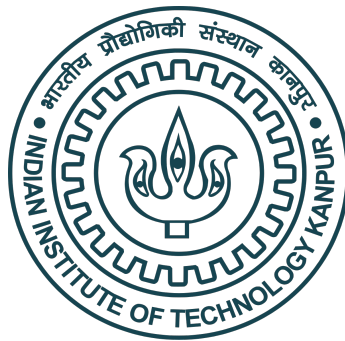

A Comparative Study of PIR Protocols Based on Secret Sharing Techniques

*A Project Submitted
In Partial Fulfillment of the Requirements
for the Degree of **Master of Technology***

Krishna Kumar Bais

(Roll No.: 241110038)



to the

**Department of Computer Science and Engineering,
Indian Institute of Technology Kanpur.**

June, 2026

Certificate

It is certified that the work contained in this project report titled “**A Comparative Study of PIR Protocols Based on Secret Sharing Techniques**”, by **Krishna Kumar Bais (Roll No. 241110038)** has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.



Prof. Adithya Vadapalli

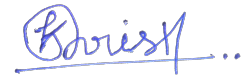
Department of Computer Science and Engineering,
Indian Institute of Technology Kanpur.

June, 2026

Declaration

This is to certify that the project report titled “**A Comparative Study of PIR Protocols Based on Secret Sharing Techniques**” has been authored by me. It presents the work carried out by me under the supervision of **Prof. Adithya Vadapalli**.

To the best of my knowledge, it is an original work, both in terms of technical content and narrative, and has not been submitted elsewhere, in part or in full, for a degree. Further, due credit has been attributed to the relevant prior art and sources through appropriate citations and acknowledgements, in line with established academic norms and practices.



Krishna Kumar Bais

Roll No. 241110038

Master of Technology

Department of Computer Science and Engineering,

Indian Institute of Technology Kanpur,

Kanpur, 208016, India.

Abstract

Name of the Student: Krishna Kumar Bais **Roll No:** 241110038
Degree for which submitted: Master of Technology **Department:** CSE
Project Title: A Comparative Study of PIR Protocols Based on Secret Sharing Techniques
Supervisor: Prof. Adithya Vadapalli
Month and Year of Submission: June, 2026

Private Information Retrieval (PIR) allows a client to fetch the k -th record of a database held by one or more servers without revealing the index k . This report studies and compares four multi-server, information-theoretic PIR constructions that differ only in how the client's index is hidden: additive secret sharing, Shamir's threshold secret sharing, a two-server scheme built on a Distributed Point Function (DPF), and an $(2, \ell)$ -server IT-PIR scheme that combines several DPFs. All four are implemented in Python over prime fields and benchmarked on query-creation time, server-computation time, reconstruction time, and upload bandwidth across database sizes from 100 to 10,000 records.

The report develops the necessary background, describes the design and implementation of each scheme, and analyses the measured results. The comparison reveals clear and complementary trade-offs: the DPF-based schemes generate queries in near-constant sub-millisecond time and the tree-compressed $(2, \ell)$ -IT-PIR scheme achieves by far the lowest, logarithmically growing upload bandwidth, while the linear secret-sharing schemes keep server computation low and are the simplest. These observations match the theoretical expectations for each family and clarify which construction is preferable under different constraints on computation and communication.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, **Prof. Adithya Vadapalli**, of the Department of Computer Science and Engineering, IIT Kanpur, for his invaluable guidance, encouragement, and insightful feedback throughout this project. His clear explanations of private information retrieval, secret sharing, and function secret sharing shaped both the implementation and the analysis presented here.

I am also thankful to the Department of Computer Science and Engineering, IIT Kanpur, for providing the resources and environment that made this work possible, and to my family and friends for their constant support.

Krishna Kumar Bais

Contents

Acknowledgements	v
List of Figures	viii
List of Tables	ix
Abbreviations	x
Symbols	xi
1 Introduction	1
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Objectives	3
1.4 Scope and Contributions	3
1.5 Report Organisation	4
2 Background and Literature Review	5
2.1 Introduction	5
2.2 Private Information Retrieval	5
2.3 Secret Sharing	6
2.3.1 Additive Secret Sharing	6
2.3.2 Shamir's Threshold Secret Sharing	7
2.3.3 Distributed Point Functions and Function Secret Sharing	7
2.4 From Building Blocks to PIR	8
2.5 Summary	8
3 System Overview and Cryptographic Building Blocks	10
3.1 Introduction	10
3.2 System Overview	10
3.3 Shamir Secret Sharing Module	12
3.4 Additive Secret Sharing Building Block	12

3.5	Distributed Point Function Module	13
3.6	Summary	14
4	PIR Protocol Designs and Implementation	15
4.1	Introduction	15
4.2	PIR via Additive Secret Sharing	15
4.3	PIR via Shamir Secret Sharing	16
4.4	Two-Server PIR from a Distributed Point Function	17
4.5	$(2, \ell)$ -Server Information-Theoretic PIR	17
4.6	Cost Comparison	18
4.7	Summary	19
5	Experimental Time Analysis	20
5.1	Introduction	20
5.2	Benchmarking Methodology	20
5.3	Results	21
5.4	Analysis	22
5.4.1	Query Creation	22
5.4.2	Server Computation	23
5.4.3	Reconstruction	23
5.4.4	Bandwidth	24
5.5	Discussion	24
5.6	Summary	24
6	Conclusion and Future Work	26
6.1	Conclusion	26
6.2	Future Work	27
	References	28

List of Figures

3.1	Repository structure showing the main implementation components, their dependencies, and the benchmark aggregator.	11
3.2	DPF evaluation tree.	14
4.1	Key assignment in $(2, \ell)$ -IT-PIR.	18

List of Tables

3.1	Summary of the implemented modules and the field used by each. . .	12
4.1	Asymptotic cost of the four PIR protocols. N is the number of records, ℓ the number of servers, t the Shamir threshold, and L the record size.	19
5.1	Measured performance at $N = 100$ records.	21
5.2	Measured performance at $N = 1000$ records.	22
5.3	Measured performance at $N = 5000$ records.	22
5.4	Measured performance at $N = 10,000$ records.	22
5.5	Best protocol on each criterion, across the tested database sizes. . . .	25

Abbreviations

PIR	P riate I nformation R etrieval
IT-PIR	I nformation- T heoretic PIR
SS	S ecret S haring
SSS	S hamir's S ecret S haring
DPF	D istributed P oint F unction
FSS	F unction S ecret S haring
PRG	P seudorandom G enerator
CW	C orrection W ord
DB	D atabase

Symbols

N	database size (number of records)
k	secret query index, $0 \leq k < N$
ℓ	number of servers
t	reconstruction threshold (Shamir SS)
p	prime modulus
n	number of bits to represent the domain
d	number of DPF instances per server (Efficient IT-PIR), $d = \log_2 \ell$
α	secret target index in a DPF
β	secret output value at α in a DPF
e_k	standard unit vector with 1 at position k
f_α^β	point function: $f_\alpha^\beta(x) = \beta$ if $x = \alpha$, else 0
L	record size (field elements per record)
$\mathbb{Z}_p$	integers modulo prime p
$\oplus$	bitwise exclusive-or (XOR)

Dedicated to my beloved parents.

Chapter 1

Introduction

1.1 Introduction

Modern services routinely answer queries against large databases: a user looks up a stock price, a patent record, a DNS entry, a movie, or a contact in a directory. In the ordinary client–server model the server learns exactly which record each client asks for. For many applications this leakage is itself sensitive, because the *access pattern*—which item a client reads—can reveal a person’s health condition, financial position, location, or intentions, even when the records themselves are public. *Private Information Retrieval* (PIR) is the cryptographic primitive that removes this leakage. It lets a client retrieve the k -th record of a database while provably hiding the index k from the server or servers that store the data.

The notion of PIR was introduced by Chor, Goldreich, Kushilevitz and Sudan [1]. Their central observation is that perfect privacy is impossible to achieve cheaply against a single server in the information-theoretic setting: the only way for one server to answer without learning anything about k is to touch the entire database and effectively send it back, which is no better than downloading everything. To escape this barrier they replicate the database across two or more *non-colluding* servers. The client then splits its query into pieces, sends one piece to each server, and combines the servers’ answers locally. As long as the servers do not pool their views, no single server learns the index. This multi-server, information-theoretic model is the setting of the present report.

A complementary line of work shows how to make the client’s query pieces extremely compact. *Secret sharing*, introduced by Shamir [2], is the classical tool for splitting a value into shares so that authorised subsets can reconstruct it and unauthorised subsets learn nothing. More recently, *distributed point functions* (DPFs) of Gilboa and Ishai [3] and the broader framework of *function secret sharing* (FSS) of Boyle, Gilboa and Ishai [4] compress a secret-shared “selection vector” into short cryptographic keys whose size grows only logarithmically with the database. These approaches—classical secret sharing and function secret sharing based on distributed point functions—give two very different ways to build the query in a multi-server PIR scheme, and comparing them is the subject of this work.

1.2 Problem Statement

A PIR scheme can be built in several ways, and the choices interact in non-obvious ways with cost. The client’s index can be hidden using additive secret sharing, using Shamir’s threshold secret sharing, or using one or more distributed point functions. Each choice fixes how the query is generated, how much data the client uploads, how much work each server must do, and how the answer is reconstructed. The theoretical literature describes these trade-offs asymptotically, but the constant factors and the cross-over points that decide which scheme is best in practice are easy to lose sight of.

This report addresses a concrete and practical question: *for the same retrieval task, how do PIR schemes based on additive secret sharing, Shamir secret sharing, and distributed point functions compare when each is actually implemented and measured?* Concretely, all four schemes are implemented over prime fields in Python, run on the same database sizes and a common database model, and benchmarked on the same four cost metrics—query-creation time, server-computation time, reconstruction time, and upload bandwidth—across a range of database sizes. The goal is to make the qualitative differences between the families quantitative and reproducible.

1.3 Objectives

The objectives of the project are the following.

- To study the three cryptographic building blocks relevant to multi-server PIR—additive secret sharing, Shamir’s threshold secret sharing, and distributed point functions—and to implement the latter two as standalone, testable modules.
- To design and implement four multi-server PIR protocols on a common database model: PIR via additive secret sharing, PIR via Shamir secret sharing, a two-server PIR built directly on a DPF, and an $(2, \ell)$ -server information-theoretic PIR that combines several DPFs.
- To build a uniform benchmarking harness that measures query-creation time, server-computation time, reconstruction time, and upload bandwidth for every protocol, and that aggregates the measurements into a single comparison table.
- To analyse the measured results, relate them to the expected asymptotic behaviour of each scheme, and identify which construction is preferable under different constraints.

1.4 Scope and Contributions

The scope of this work is deliberately focused. All schemes operate in the *information-theoretic, multi-server* model with honest-but-curious, non-colluding servers; computational single-server PIR based on homomorphic encryption is outside the scope. The implementations are reference implementations written for clarity and measurement rather than for production hardening, and the servers are simulated as local processes rather than networked machines, so the reported times reflect computation and local data handling rather than network latency. The complete source code described here is publicly available at https://github.com/krishna-kumar-bais/PIR_and_Secret-Sharing.

Within this scope, the contributions of the report are: (i) clean, self-contained implementations of Shamir secret sharing and of a distributed point function, reused

as building blocks; (ii) four PIR protocols built on a shared database and server model so that they can be compared fairly; (iii) a single benchmarking harness and an aggregator that produces a uniform, per-database-size comparison; and (iv) an analysis of the resulting measurements that explains the observed trade-offs and connects them to the underlying theory.

1.5 Report Organisation

This report consists of six chapters, including the present introduction (Chapter 1), which sets out the motivation, problem, objectives, and scope. Chapter 2 reviews the background and prior work on private information retrieval, secret sharing. Chapter 3 gives an overview of the implemented system and describes the two cryptographic building blocks—Shamir secret sharing and the distributed point function—in detail. Chapter 4 presents the design and implementation of the four PIR protocols and discusses their cost structure. Chapter 5 describes the benchmarking methodology and presents and analyses the measured results. Chapter 6 summarises the findings, states the conclusions, and outlines directions for future work.

Chapter 2

Background and Literature Review

2.1 Introduction

This chapter develops the background needed for the rest of the report. It first defines private information retrieval and explains why the multi-server, information-theoretic model is used. It then reviews the two ways the client’s query is built in this project—secret sharing (additive and Shamir’s threshold scheme) and distributed point functions—and finally explains how these primitives are combined to obtain a working PIR protocol.

2.2 Private Information Retrieval

A database is modelled as a vector $DB = (DB[0], DB[1], \dots, DB[N-1])$ of N records. A client wishes to learn $DB[k]$ for some secret index k without revealing k to whoever holds the data. A trivial solution is to download the whole database; PIR aims to do much better while keeping k private.

Chor, Goldreich, Kushilevitz and Sudan [1] formalised this problem and proved a fundamental limitation: with a single server and information-theoretic privacy, any scheme must communicate essentially the entire database, so non-trivial single-server PIR is impossible in this model. Their way around the obstacle is *replication across non-colluding servers*. The database is copied to $\ell \geq 2$ servers that are assumed

not to share their views of the protocol. The client sends a different query to each server; each query, examined alone, is independent of k and therefore reveals nothing. The servers reply, and the client combines the replies to recover $\text{DB}[k]$. Because privacy follows from the assumption that the servers do not collude rather than from any computational hardness, the guarantee is *information-theoretic*: it holds even against a computationally unbounded server.

A clean and widely used way to instantiate this idea is the *linear* or *inner-product* template. The records are treated as elements of a finite field $\mathbb{Z}_p$. Reading index k is the same as computing the inner product $\langle e_k, \text{DB} \rangle$, where e_k is the unit vector that is 1 at position k and 0 elsewhere. To hide k , the client splits the selection vector e_k into shares, one per server; each server returns the inner product of its share with the database; and the client combines the returned values. The privacy of the scheme therefore reduces to the privacy of the way e_k is shared, and the server's cost is one pass over the database. All four protocols in this report follow this template and differ only in how the selection vector is shared and reconstructed.

2.3 Secret Sharing

Secret sharing splits a secret into shares distributed among parties so that authorised subsets can reconstruct the secret while unauthorised subsets learn nothing about it. Two variants are used in this project.

2.3.1 Additive Secret Sharing

In additive sharing over $\mathbb{Z}_p$, a secret s is split among ℓ parties by choosing the first $\ell - 1$ shares uniformly at random and setting the last share so that all shares sum to s modulo p . Any $\ell - 1$ shares are jointly uniform and reveal nothing about s ; only the full set of ℓ shares reconstructs it, by addition. Additive sharing is the simplest scheme and is naturally suited to the inner-product template: sharing each coordinate of e_k additively yields ℓ random-looking query vectors whose coordinate-wise sum is exactly e_k , so the sum of the servers' inner products is $\text{DB}[k]$.

2.3.2 Shamir's Threshold Secret Sharing

Shamir's scheme [2] introduces a *threshold* t : any t shares reconstruct the secret, but any $t - 1$ shares reveal nothing. To share a secret $s \in \mathbb{Z}_p$, one samples a random polynomial of degree $t - 1$ whose constant term is s and evaluates it at ℓ distinct non-zero points; the share of party i is the value of the polynomial at its point. Reconstruction from any t shares is Lagrange interpolation, which recovers the constant term s . The security follows from the fact that fewer than t evaluations of a degree- $(t - 1)$ polynomial leave its constant term uniformly distributed. Shamir sharing generalises additive sharing (which is the $t = \ell$ case in spirit) and, when used to share each coordinate of e_k , yields a threshold PIR scheme in which the client can reconstruct from any t of the ℓ server responses and tolerate collusion of up to $t - 1$ servers.

2.3.3 Distributed Point Functions and Function Secret Sharing

The selection vector e_k is a special case of a *point function*: the function $f_{\alpha,\beta}$ that evaluates to β at a single input α and to 0 on all other inputs (here $\alpha = k$ and $\beta = 1$). A *distributed point function* (DPF), introduced by Gilboa and Ishai [3], secret-shares such a function rather than a single value. It produces two short keys k_0, k_1 such that, for every input x , the two parties' local evaluations sum to $f_{\alpha,\beta}(x)$, while a single key hides both α and β . The decisive property is succinctness: instead of sharing all N coordinates of e_k explicitly, the client sends keys whose size is proportional to the *bit-length* of the domain, that is, logarithmic in N .

Modern DPF constructions realise this with a binary tree of pseudorandom seeds. Starting from a random root seed and a one-bit control flag, each party uses a pseudorandom generator (PRG) to expand a node into left and right children, and a per-level *correction word* is published so that the two parties' states stay identical on every node off the special path to α and diverge in a controlled way on the path itself. A final correction word fixes the output value to β at the leaf α . Because the parties agree off the path, their leaf shares cancel everywhere except at α , which is exactly the point-function property. Boyle, Gilboa and Ishai later generalised DPFs to *function secret sharing* [4], which shares richer classes of functions (intervals,

comparisons, and more) in the same succinct, key-based manner, and which underlies many subsequent efficient PIR and secure-computation systems.

DPFs are directly useful for PIR. A two-server scheme has the client generate a DPF for $\alpha = k, \beta = 1$, send one key to each server, and have each server return the inner product of its full evaluation vector with the database; summing the two responses yields $\text{DB}[k]$, with upload cost logarithmic in N rather than linear. The construction can be lifted to more servers: by combining $d = \log_2 \ell$ independent DPF instances and assigning keys to servers according to their binary labels, one obtains an $(2, \ell)$ -server information-theoretic scheme that further reduces communication while distributing trust across ℓ servers.

2.4 From Building Blocks to PIR

The two families lead to PIR schemes with complementary cost profiles, which is precisely what this report measures. Secret-sharing schemes share the selection vector coordinate by coordinate, so the client produces ℓ query vectors of length N ; query creation and upload bandwidth therefore grow linearly with N , but each server's work is a single, cache-friendly linear scan and reconstruction is a short combination of ℓ field elements. DPF-based schemes instead send short keys, so query creation is near-constant and upload bandwidth grows only logarithmically with N ; the cost is shifted to the servers, which must expand the key over the whole domain—one PRG-driven tree evaluation per record—before taking the inner product. The remainder of this report makes these qualitative statements concrete by implementing all four schemes on a common database model and benchmarking them under identical conditions.

2.5 Summary

This chapter defined private information retrieval, explained why this work adopts the multi-server, information-theoretic model of Chor et al. [1], and reviewed the building blocks used to hide the client's index: additive and Shamir [2] secret sharing, and distributed point functions [3] within the function secret sharing framework [4]. It also explained the common inner-product template that lets all four protocols be

compared on equal footing. The next chapter describes the implemented system and the building-block modules in detail.

Chapter 3

System Overview and Cryptographic Building Blocks

3.1 Introduction

This chapter describes the implemented system as a whole and then explains, in detail but without reproducing source code, the two cryptographic building blocks that the PIR protocols reuse: Shamir’s secret-sharing module and the distributed point function module. The protocols themselves are the subject of Chapter 4, and their measured performance is reported in Chapter 5.

3.2 System Overview

The codebase is organised as standalone primitive modules, PIR protocol implementations, time-analysis variants, and a top-level benchmark aggregator, as shown in Figure 3.1. There are two *standalone* secret-sharing modules—Shamir’s secret sharing and a distributed point function—which exist so that each primitive can be exercised and validated on its own. There are four *PIR protocols* that reuse these primitives: PIR via additive secret sharing, PIR via Shamir secret sharing, a two-server PIR built directly on a DPF, and an $(2, \ell)$ -server information-theoretic PIR. Each protocol has a parallel *time-analysis variant* that instruments the same logic with timing and bandwidth measurements, and all four variants are driven by

a single aggregator script, `run_all_time_analysis.py`, which runs them over a list of database sizes and writes a combined results file. The dashed “reuse” edges in the figure record the dependency of `PIR_with_SSS` on the Shamir module and of the two DPF-based protocols on the DPF module.

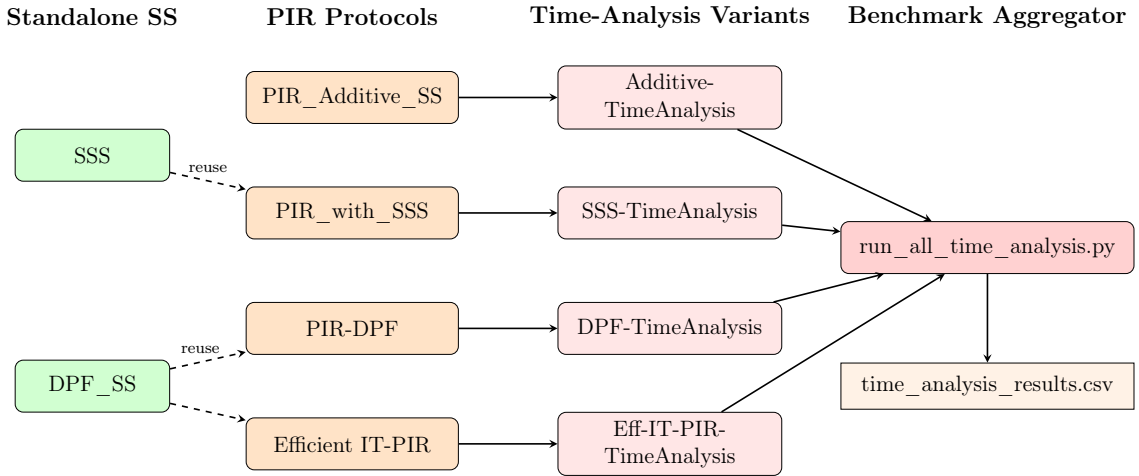


FIGURE 3.1: Repository structure showing the main implementation components, their dependencies, and the benchmark aggregator.

All modules follow a common design so that the protocols can be compared fairly. A database is a list of N integer records; in the secret-sharing protocols each record is a single field element drawn at random, while in the DPF-based $(2, \ell)$ scheme each record is a tuple of L field elements, which lets the scheme retrieve a multi-element record in one query. Arithmetic is carried out modulo a prime, so the records live in a finite field $\mathbb{Z}_p$. Two Mersenne primes are used in the implementation: the secret-sharing and two-server DPF protocols accumulate modulo $2^{61} - 1$, while the standalone demonstrations and the $(2, \ell)$ -IT-PIR scheme use $2^{31} - 1$. In the additive, Shamir, and two-server DPF protocols the servers are simulated as worker processes in a process pool, so their inner-product computations run in parallel on separate CPU cores; the $(2, \ell)$ -IT-PIR scheme instead evaluates its servers sequentially within a single process. In all cases the timings reflect local computation rather than network transfer. Table 3.1 summarises the modules, their roles, and the field each one uses.

TABLE 3.1: Summary of the implemented modules and the field used by each.

Module	Role	Field $\mathbb{Z}_p$
SSS	Standalone Shamir (t, n) secret sharing demo	$2^{31} - 1$
DPF_SS	Standalone distributed point function demo	$2^{31} - 1$
PIR_Additive_SS	PIR via additive secret sharing	$2^{61} - 1$
PIR_with_SSS	PIR via Shamir secret sharing	$2^{61} - 1$
PIR-DPF	Two-server PIR built on a DPF	$2^{61} - 1$
Efficient_IT_PIR	$(2, \ell)$ -server IT-PIR from $\log_2 \ell$ DPFs	$2^{31} - 1$

3.3 Shamir Secret Sharing Module

The Shamir module provides the two operations needed by threshold secret sharing: share generation and reconstruction. To share a secret s with threshold t among n parties, the generator builds a polynomial of degree $t - 1$ whose constant coefficient is s and whose remaining $t - 1$ coefficients are drawn uniformly at random from the field. It then evaluates this polynomial at the points $x = 1, 2, \dots, n$ using Horner-style repeated multiplication modulo the prime, and returns the n pairs $(x, \text{polynomial}(x))$ as the shares. Because the constant term is the secret, the value at $x = 0$ is s ; because the other coefficients are random, any fewer than t shares leave s uniformly distributed.

Reconstruction takes any t of the shares and recovers the secret by Lagrange interpolation evaluated at the origin. For each chosen share the module computes the Lagrange coefficient as a product of ratios of the other shares' x -coordinates, performing the division as multiplication by a modular inverse so that all arithmetic stays in the field. The secret is then the weighted sum of the chosen y -values with these coefficients. The standalone module demonstrates the full cycle—splitting a sample secret into $n = 5$ shares with threshold $t = 3$ and reconstructing it from a subset—and the same share-generation and reconstruction routines are reused unchanged inside the Shamir-based PIR protocol.

3.4 Additive Secret Sharing Building Block

Additive sharing is used directly inside the additive-PIR protocol rather than as a separate module. To split a value among ℓ parties, the first $\ell - 1$ shares are

sampled uniformly at random from the field and the last share is set so that all shares sum to the value modulo the prime. Reconstruction is simply the modular sum of all ℓ shares. This is the threshold- ℓ special case of secret sharing: every share is required to reconstruct, and any proper subset is jointly uniform and therefore reveals nothing.

3.5 Distributed Point Function Module

The DPF module implements the two-key point-function sharing described in Chapter 2, following the tree-based construction. Its pseudorandom generator is built from SHA-256: given a seed it hashes the seed concatenated with a counter and concatenates the digests until enough output bytes are produced. Seeds are 16 bytes; expanding a seed yields enough material for a left child seed, a right child seed, and two one-bit control flags. A helper maps a node seed to a field element by hashing it and reducing modulo the prime.

Key generation. Generation begins from two random root seeds and two complementary root control bits, one for each party. It then walks the bits of the target index α from the most significant to the least significant. At each level both parties expand their current seed and parse the result into left and right child seeds and flags. The level's *correction word* consists of a seed correction, computed as the bitwise XOR of the two parties' child seeds on the branch that is *not* on the path to α , together with two one-bit corrections for the left and right control flags. Each party then advances to the child indicated by the current bit of α , applying the seed correction only when its control bit is set, and updates its control bit using the published flag corrections. The effect is that the two parties' seeds and control bits remain identical on every node off the path to α and differ on the path itself. After the last level a final output correction word is computed from the field images of the two parties' leaf seeds so that the parties' outputs sum to β at the leaf α . Each key consists of the party's root seed and control bit, the list of per-level correction words, and the output correction word.

Evaluation. To evaluate a key at an input x , a party starts from the key's root seed and control bit and walks the bits of x . At each level it expands the current seed, and if its control bit is set it XORs in the level's seed correction and applies the flag

corrections, then descends into the child selected by the current bit of x , carrying that child’s seed and control bit forward. At the leaf the party maps its seed to a field element, adds the output correction word if its control bit is set, and negates the result for the second party. By construction, the two parties’ outputs cancel for every $x \neq \alpha$ and sum to β at $x = \alpha$, which is exactly the point-function property. Figure 3.2 illustrates the evaluation tree and the special path. The standalone module demonstrates this by generating keys for a sample (α, β) and printing, for each input in a small domain, the two parties’ shares and their sum, which is non-zero only at α .

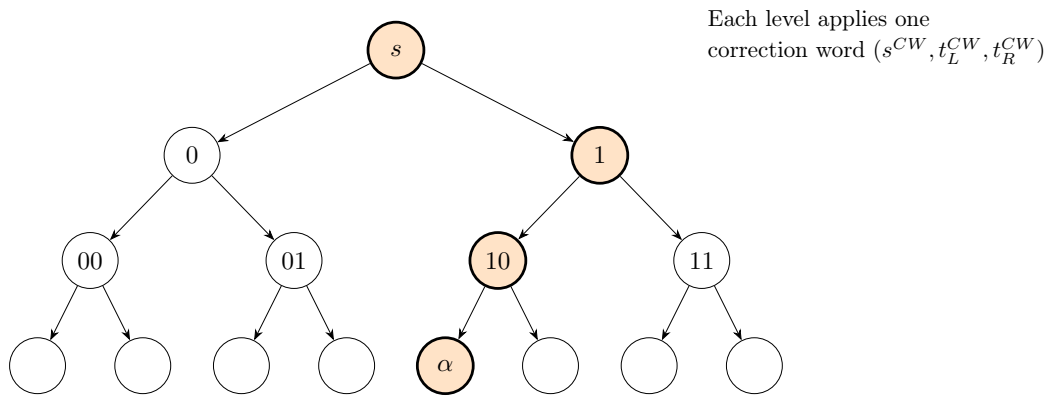


FIGURE 3.2: DPF evaluation tree of depth $n = 3$ for target index α . Each party walks the binary expansion of the input from the root, expanding a node’s seed with the PRG and applying the level correction word only when its control bit is set. The two parties’ seeds and control bits agree off the highlighted path (so their shares cancel) and differ on it (so their shares sum to β).

3.6 Summary

This chapter described the overall structure of the system—standalone primitives, PIR protocol implementations, time-analysis variants, and an aggregator built on a common database and field model—and then explained the two reusable building blocks in detail: the Shamir secret-sharing module, with its polynomial-evaluation share generation and Lagrange reconstruction, and the distributed point function module, with its SHA-256-based PRG, per-level correction words, and leaf output correction. The next chapter shows how these blocks, together with additive sharing, are assembled into the four PIR protocols.

Chapter 4

PIR Protocol Designs and Implementation

4.1 Introduction

All four protocols implement the inner-product template of Chapter 2: the client conceals the unit selection vector e_k , each server returns the inner product of its share of e_k with the database, and the client combines the responses to obtain $\text{DB}[k]$. The protocols differ only in how e_k is shared, how many servers are involved, and how the responses are combined. This chapter describes each protocol in turn—its query generation, server computation, reconstruction, and privacy—and then compares their cost structures.

4.2 PIR via Additive Secret Sharing

In the additive scheme the client shares the selection vector across ℓ servers coordinate by coordinate. It draws the query vectors of the first $\ell - 1$ servers as fully random length- N vectors over the field, and constructs the last server's vector so that, position by position, the ℓ vectors sum to e_k : at the queried position the coordinates sum to 1, and at every other position they sum to 0. Each server returns the inner product of its query vector with the database, and the client sums the ℓ

responses modulo the prime. Because the query vectors sum to e_k , this sum equals $\text{DB}[k]$.

Privacy follows directly from additive sharing: any $\ell - 1$ of the query vectors are jointly uniform and independent of k , so no coalition short of all servers learns anything about the index. The scheme is the simplest of the four; query creation and upload bandwidth grow linearly with N because the client materialises ℓ full-length vectors, while each server performs a single linear scan of the database and reconstruction is just the sum of ℓ field elements.

4.3 PIR via Shamir Secret Sharing

The Shamir-based scheme replaces additive sharing with threshold sharing so that the client need not wait for every server. For each coordinate of the selection vector the client takes the secret to be 1 at the queried position and 0 elsewhere, and shares that bit among the ℓ servers using a fresh degree- $(t - 1)$ polynomial, as described in Chapter 3. Server i thus receives the evaluations at its point of one polynomial per coordinate. When server i computes the inner product of its query vector with the database, linearity of evaluation means the result is the value at server i 's point of a single polynomial of degree $t - 1$ whose constant term is exactly $\text{DB}[k]$. The servers' responses are therefore threshold shares of the desired record.

The client reconstructs $\text{DB}[k]$ by Lagrange interpolation at the origin. A polynomial of degree $t - 1$ is fixed by any t of the responses, so t responses are sufficient in principle; the implementation interpolates from $t + 1$ responses—one more than strictly required—which yields the same constant term. The threshold makes the scheme robust: it tolerates the collusion of up to $t - 1$ servers without leaking k and can reconstruct even if some servers fail to respond. The price relative to additive sharing is heavier query creation, because each of the N coordinates requires evaluating a random polynomial at ℓ points; upload bandwidth remains linear in N , server computation is again a single scan, and reconstruction is a short Lagrange combination.

4.4 Two-Server PIR from a Distributed Point Function

The two-server DPF scheme abandons explicit length- N query vectors in favour of the succinct keys of Chapter 3. The client generates a single DPF for the point function that is 1 at the queried index and 0 elsewhere, using enough bits to represent all database positions, and sends one key to each of the two servers. Each server evaluates its key at every database position to obtain its share of the selection vector, multiplies these shares into the database, and accumulates the inner product modulo the prime. The client adds the two servers' responses to recover $\text{DB}[k]$, because the two evaluations sum to the selection vector pointwise.

The advantage is communication: a DPF key encodes the entire selection vector in space proportional to the logarithm of the domain, so query creation is near-constant and the client's true upload is tiny and grows only logarithmically with N . The cost moves to the servers, each of which must run one PRG-driven tree evaluation per record before forming the inner product, making server work grow slightly faster than a plain linear scan. A single key hides both the index and the value, so neither server alone learns anything about k ; privacy again relies on the two servers not colluding.

4.5 $(2, \ell)$ -Server Information-Theoretic PIR

The $(2, \ell)$ -IT-PIR scheme generalises the two-server idea to ℓ servers, where ℓ is a power of two, and supports records made of L field elements. The client first computes the number of DPF levels $d = \log_2 \ell$ and generates d *independent* DPF key pairs, all for the same queried index. It then labels the servers with d -bit binary strings and gives server b , level by level, the key of pair i selected by the i -th bit of b , as illustrated in Figure 4.1. Each server evaluates each of its d keys over the whole domain and, for every key, returns the inner product of the resulting selection vector with the database; since records have L elements, every such answer is itself an L -element vector.

To reconstruct, the client finds any two responding servers whose labels differ, locates the lowest bit position i in which they differ, and adds those two servers' i -th answer

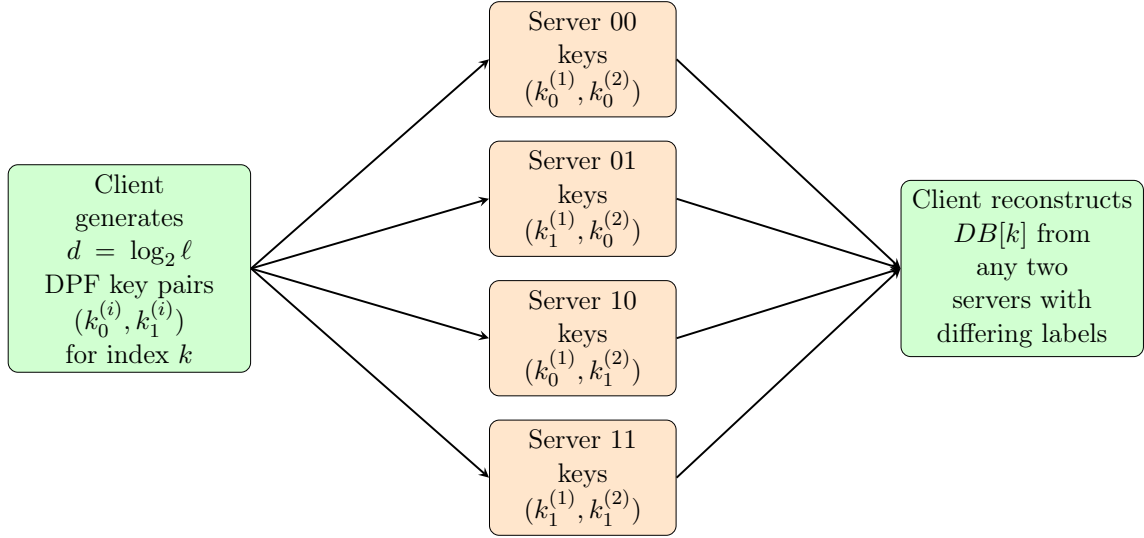


FIGURE 4.1: Key assignment in the $(2, \ell)$ -IT-PIR scheme for $\ell = 4$ servers ($d = 2$ DPF levels). The client builds d independent DPF key pairs for the same index k and gives server b the key selected, level by level, by the bits of its binary label b . Any two servers whose labels differ in some bit hold opposite keys at that level, so their answers reconstruct the record.

vectors coordinate by coordinate. At that level the two servers were given opposite keys of the same DPF pair, so their answers sum to the record; at the other levels they were given identical keys, which is what allows the client to single out the differing level. The scheme therefore reconstructs as long as at least two servers with distinct labels respond, so it tolerates the loss of up to $\ell - 2$ servers, and a single server's keys reveal nothing about the index. Its decisive advantage is bandwidth: only short DPF keys are sent, and because the keys are tree-compressed the total upload grows only logarithmically with N , which Chapter 5 confirms is by far the smallest of the four schemes.

4.6 Cost Comparison

Table 4.1 summarises the asymptotic cost of the four protocols in terms of the database size N , the number of servers ℓ , and (for Shamir) the threshold t . The secret-sharing schemes pay linear query and communication cost but keep the per-server computation to a single field-multiply-and-add scan, whereas the DPF schemes invert this trade-off, paying logarithmic communication but a PRG tree evaluation per record. These predictions are tested empirically in the next chapter.

TABLE 4.1: Asymptotic cost of the four PIR protocols. N is the number of records, ℓ the number of servers, t the Shamir threshold, and L the record size.

Protocol	Servers	Query gen.	Upload	Server work	Collusion
PIR_Additive_SS	$\ell \geq 2$	$O(\ell N)$	$O(\ell N)$	$O(N)$	up to $\ell - 1$
PIR_with_SSS	ℓ	$O(\ell N t)$	$O(\ell N)$	$O(N)$	up to $t - 1$
PIR-DPF	2	$O(\log N)$	$O(\log N)$	$O(N \log N)$	requires non-collusion
Efficient_IT_PIR	ℓ (pow. 2)	$O(\log \ell \log N)$	$O(\ell \log \ell \log N)$	$O(\ell \log \ell N \log N)$	requires non-collusion

4.7 Summary

This chapter described the four PIR protocols on the common inner-product template: the additive scheme that sums all server responses, the Shamir scheme that reconstructs from a threshold number of responses and tolerates collusion, the two-server DPF scheme that shrinks the query to a single succinct key, and the $(2, \ell)$ -IT-PIR scheme that combines $\log_2 \ell$ DPFs to spread trust across many servers while keeping communication logarithmic. The cost comparison made the expected trade-off explicit; the following chapter measures it.

Chapter 5

Experimental Time Analysis

5.1 Introduction

This chapter reports and analyses the measured performance of the four PIR protocols. The aim is to turn the qualitative trade-offs of Chapter 4 into concrete numbers and to check whether the implementations behave as the theory predicts. All figures quoted here are the measurements recorded in the project’s benchmark output and comparison files.

5.2 Benchmarking Methodology

Each protocol is instrumented by a time-analysis variant that reproduces the protocol’s logic and measures three phases separately—query creation, server computation, and reconstruction—together with the upload bandwidth from the client to the servers. The aggregator script `run_all_time_analysis.py` runs all four variants over a common list of database sizes, parses their output, and writes a single combined table per database size. The four database sizes used are $N = 100, 1000, 5000$, and 10,000 records, and the queried index is $k = 0$.

The protocols are run with their default configurations: the additive and Shamir schemes use three servers (with Shamir threshold $t = 2$), the DPF scheme uses two servers, and the $(2, \ell)$ -IT-PIR scheme uses four servers with single-element records.

In the additive, Shamir, and two-server DPF protocols the servers run as worker processes in a process pool, so the server-computation phase includes the fixed cost of creating and feeding that pool and reports the wall-clock time of the parallel work. The $(2, \ell)$ -IT-PIR scheme, by contrast, evaluates its servers sequentially in a single process, so its server-computation figure is the *sum* of all servers' work and carries no pool-startup cost; this difference should be kept in mind when comparing server times across the two groups. The $(2, \ell)$ -IT-PIR measurements are averaged over ten runs; the other three protocols are measured once per size, so their figures are single-run measurements for which the run-to-run variance was not estimated and should not be read as zero. Times are reported in milliseconds and bandwidth in kilobytes.

Two points about the bandwidth figures should be kept in mind when comparing across schemes. For the secret-sharing protocols the harness estimates upload as the in-memory size of the query objects, which scales linearly with N ; for the $(2, \ell)$ -IT-PIR scheme it sums the actual serialized sizes of the DPF keys, which are tree-compressed and grow only logarithmically; and for the two-server DPF scheme it models the upload as two fixed-size keys *per record*, which makes its reported bandwidth scale linearly even though a genuine DPF key is logarithmic in N . The bandwidth column is therefore best read as a faithful record of each implementation's accounting and of how each scheme *scales*, rather than as an exact count of wire bytes.

5.3 Results

Tables 5.1 through 5.4 give the measured averages for the four database sizes.

TABLE 5.1: Measured performance at $N = 100$ records.

Protocol	Query (ms)	Server (ms)	Recon. (ms)	Bandwidth (KB)
PIR_Additive_SS	0.192	192.637	0.005	2.70
PIR_with_SSS	0.291	181.899	0.020	2.51
PIR-DPF	0.084	187.825	0.002	12.50
Efficient_IT_PIR	0.133	22.272	0.006	1.19

TABLE 5.2: Measured performance at $N = 1000$ records.

Protocol	Query (ms)	Server (ms)	Recon. (ms)	Bandwidth (KB)
PIR_Additive_SS	1.659	151.705	0.006	25.95
PIR_with_SSS	2.654	153.169	0.025	23.60
PIR-DPF	0.108	189.149	0.004	125.00
Efficient_IT_PIR	0.170	251.039	0.005	1.61

TABLE 5.3: Measured performance at $N = 5000$ records.

Protocol	Query (ms)	Server (ms)	Recon. (ms)	Bandwidth (KB)
PIR_Additive_SS	8.565	154.121	0.029	122.70
PIR_with_SSS	39.593	166.688	0.018	117.35
PIR-DPF	0.132	360.888	0.004	625.00
Efficient_IT_PIR	0.214	2556.471	0.005	2.03

TABLE 5.4: Measured performance at $N = 10,000$ records.

Protocol	Query (ms)	Server (ms)	Recon. (ms)	Bandwidth (KB)
PIR_Additive_SS	16.959	154.955	0.004	249.54
PIR_with_SSS	26.490	155.381	0.018	234.54
PIR-DPF	0.126	602.542	0.003	1250.00
Efficient_IT_PIR	0.229	5658.742	0.006	2.17

5.4 Analysis

5.4.1 Query Creation

The two DPF-based schemes generate their queries in well under a millisecond and the time barely changes as N grows—from 0.084 to 0.126 ms for the two-server DPF and from 0.133 to 0.229 ms for $(2, \ell)$ -IT-PIR. This is exactly what the theory predicts: a DPF key depends on the *bit-length* of the index, not on the number of records, so enlarging the database leaves key generation essentially untouched. The secret-sharing schemes behave oppositely. Their query creation grows with N because the client materialises a full-length share vector for every server: the additive scheme rises from 0.192 ms at $N = 100$ to 16.959 ms at $N = 10,000$, and the Shamir scheme is consistently the heaviest of the two because it must evaluate a random polynomial at every server’s point for each of the N coordinates. The large Shamir value at $N = 5000$ (39.593 ms) is inflated by run-to-run noise—these three protocols

are measured only once per size—but the linear trend and the polynomial-evaluation overhead are clear.

5.4.2 Server Computation

Server computation is where the schemes diverge most sharply. For the additive and Shamir schemes the measured server time stays almost flat near 150–170 ms across all four sizes. The actual linear scan over the database is cheap at these sizes, so the measured time is dominated by the fixed cost of starting and feeding the worker-process pool rather than by the database work itself; the linear growth of the scan only begins to show through at the largest size. The DPF schemes show the opposite, and theoretically expected, behaviour: their server work grows with N because each server must expand the DPF key over the entire domain—one PRG tree evaluation per record—before forming the inner product. The two-server DPF rises from about 188 ms to 603 ms. The $(2, \ell)$ -IT-PIR scheme is the fastest of all at the smallest size (22.272 ms at $N = 100$)—because, unlike the other three, it uses no process pool and therefore avoids the fixed pool-startup cost—but it becomes the slowest by far at the largest size (5658.742 ms at $N = 10,000$), since it evaluates several full-domain DPF trees across its four servers sequentially in a single process. It should also be noted that the $(2, \ell)$ -IT-PIR scheme operates over the smaller field $\mathbb{Z}_{2^{31}-1}$, whereas the other three protocols use $\mathbb{Z}_{2^{61}-1}$; the smaller field makes each field operation cheaper, which marginally favours IT-PIR in the server-time comparison.

5.4.3 Reconstruction

Reconstruction is negligible for every protocol and every database size: all measured values are below 0.03 ms. This matches the analysis of Chapter 4, where reconstruction is at most a short combination of a handful of field elements—an additive sum, a small Lagrange interpolation, or the coordinate-wise addition of two answer vectors—none of which depends meaningfully on N .

5.4.4 Bandwidth

The bandwidth results show the clearest separation. The $(2, \ell)$ -IT-PIR scheme is by far the most communication-efficient: its upload grows only from 1.19 KB to 2.17 KB as the database scales by two orders of magnitude, reflecting the logarithmic size of tree-compressed DPF keys. The additive and Shamir schemes grow linearly, from a few kilobytes at $N = 100$ to roughly a quarter of a megabyte at $N = 10,000$, because the client must send a full-length share vector to each server. The two-server DPF scheme reports the largest upload at the larger sizes (up to 1250 KB), but, as noted in the methodology, this is an artefact of the harness modelling the upload as a fixed-size key *per record*; a true two-server DPF key is logarithmic in N , as the IT-PIR measurements demonstrate. The honest reading is that all DPF-based queries are intrinsically succinct, and the $(2, \ell)$ -IT-PIR measurements capture this directly.

5.5 Discussion

No single protocol dominates; the right choice depends on whether computation or communication is the binding constraint, and on the trust model. Table 5.5 summarises which protocol is best on each criterion. The DPF family wins decisively on query creation, reconstruction, and—for the tree-compressed $(2, \ell)$ scheme—bandwidth, which makes it attractive when the client is weak or the link is expensive. The secret-sharing family keeps server computation low and predictable and is the simplest to implement and reason about, which makes it attractive when the servers are the bottleneck or when threshold robustness and collusion tolerance are required. The $(2, \ell)$ -IT-PIR scheme offers the most appealing communication profile but pays for it with server computation that grows fastest of all, so it is best suited to small or moderate databases where its tiny upload and low server cost coincide.

5.6 Summary

The measurements confirm the predicted trade-off between the two families. DPF-based queries are generated in near-constant time and are intrinsically succinct, but shift cost onto the servers, whose work grows with N and grows fastest for the

TABLE 5.5: Best protocol on each criterion, across the tested database sizes.

Criterion	Best protocol
Fastest query creation	PIR-DPF (sub-0.2 ms, near-constant)
Lowest server computation (small N)	Efficient_IT_PIR (~ 22 ms at $N = 100$)
Lowest server computation (large N)	PIR_Additive_SS / PIR_with_SSS (~ 155 ms)
Lowest upload bandwidth (all sizes)	Efficient_IT_PIR (1.19–2.17 KB)
Fastest reconstruction	PIR-DPF (< 0.005 ms)
Simplest construction	PIR_Additive_SS

multi-server $(2, \ell)$ scheme. Secret-sharing queries grow linearly in size and creation time but keep server computation low and stable and offer threshold robustness. Reconstruction is negligible throughout. The next chapter draws these observations together into conclusions and outlines future work.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project studied multi-server, information-theoretic private information retrieval through the lens of how the client’s query is hidden, and compared four constructions on a single, uniform footing. Two cryptographic building blocks were implemented as standalone modules—Shamir’s threshold secret sharing and a distributed point function—and were reused, together with additive secret sharing, to build four PIR protocols: PIR via additive secret sharing, PIR via Shamir secret sharing, a two-server PIR built directly on a DPF, and an $(2, \ell)$ -server IT-PIR scheme that combines $\log_2 \ell$ DPFs. All four follow the same inner-product template and were benchmarked by a common harness over database sizes from 100 to 10,000 records.

The central finding is that the secret-sharing family and the DPF family occupy opposite and complementary corners of the design space, and that the measurements match the theory closely. Secret-sharing schemes share the selection vector coordinate by coordinate, so their query-creation time and upload bandwidth grow linearly with the database, but each server only performs a single linear scan and the schemes are simple and, in the Shamir case, robust to server collusion and failure. DPF-based schemes compress the query into short keys, so query creation is near-constant and—for the tree-compressed $(2, \ell)$ -IT-PIR scheme—upload bandwidth grows only logarithmically, from about 1.2 KB to 2.2 KB across the tested range; the price is server computation that grows with the database and grows fastest for the multi-server scheme. Reconstruction was negligible for every protocol. In short, the two-server

DPF scheme is the clear choice for fast, lightweight queries, the $(2, \ell)$ -IT-PIR scheme is the clear choice when communication must be minimised on small to moderate databases, and the secret-sharing schemes are preferable when server computation is the bottleneck or when threshold guarantees are required.

6.2 Future Work

Several directions would strengthen and extend this study. First, the servers are currently simulated as local processes, so the reported times capture computation rather than network behaviour; deploying the servers across real machines and including transfer cost would make the bandwidth advantages of the DPF schemes far more visible end-to-end. Second, the benchmarks could be made more rigorous by averaging every protocol over many runs (not only the $(2, \ell)$ -IT-PIR scheme), reporting confidence intervals, and extending the database sizes well beyond 10,000 records to expose the asymptotics more clearly. Third, the server-side cost of the DPF schemes could be reduced substantially by using full-domain evaluation, which amortises the tree traversal across all records, and by replacing the SHA-256-based generator with a faster pseudorandom generator. Fourth, the bandwidth accounting could be unified so that every protocol reports exact serialized wire sizes, removing the per-record modelling used for the two-server DPF scheme. Finally, the comparison could be broadened to include computational single-server PIR based on homomorphic encryption and richer function secret sharing for range and keyword queries, placing the information-theoretic schemes studied here within the wider landscape of private retrieval.

References

- [1] B. Chor, O. Goldreich, E. Kushilevitz, and M. Sudan, “Private information retrieval,” in *Proceedings of the 36th Annual Symposium on Foundations of Computer Science (FOCS)*. IEEE, 1995, pp. 41–50.
- [2] A. Shamir, “How to share a secret,” *Communications of the ACM*, vol. 22, no. 11, pp. 612–613, 1979.
- [3] N. Gilboa and Y. Ishai, “Distributed point functions and their applications,” in *Advances in Cryptology – EUROCRYPT 2014*, ser. Lecture Notes in Computer Science, vol. 8441. Springer, 2014, pp. 640–658.
- [4] E. Boyle, N. Gilboa, and Y. Ishai, “Function secret sharing,” in *Advances in Cryptology – EUROCRYPT 2015*, ser. Lecture Notes in Computer Science, vol. 9057. Springer, 2015, pp. 337–367.